JSS MAHAVIDYAPEETHA

JSS Science and Technology University

Sri Jayachamrajendra College of Engineerging

Digital Signal Processing

(22EI450)

"AUDIO SPEED CHANGER USING INTERPOLATION AND DECIMATION"

Event-II

By:

Sanjana.S.Pai (02JST23UEI055)

Pratheeksha.M (02JST23UEI047)

Prakhyath M Bharadwaj (02JST23UEI045)

Submitted to:

Ms. Rajeshwari P

Asst. Professor

Dept. of Electronics and Instrumentation,

SJCE, JSSSTU

DEPARTMENT OF ELECTRONICS AND INSTRUMENTATION ENGINEERING

2025-2026

CONTENT

- Introduction
- Objective
- Block diagram
- Explanation
- Code
- Steps for execution
- Result
- Application
- Advantages
- Disadvantages
- Conclusion

INTRODUCTION:

An Audio Speed Changer is a digital signal processing (DSP)–based system that modifies the playback speed of an audio signal using interpolation, decimation, and an FIR low-pass filter. Interpolation increases the sampling rate by inserting additional samples between existing ones, thereby expanding the signal in time and producing slower audio playback. Decimation, on the other hand, reduces the sampling rate by removing samples, which compresses the signal in time and results in faster audio playback. These multirate signal processing operations are widely used in modern audio systems for flexible speed control.

However, interpolation and decimation introduce unwanted frequency components such as spectral images and aliasing, which can degrade audio quality. To overcome these effects, an FIR low-pass filter is employed to suppress undesired frequency components while preserving the useful signal band. FIR filters are especially suitable for audio applications due to their linear phase response, which ensures minimal waveform distortion and maintains natural sound quality.

In this project, the audio speed changer is implemented as a real-time system using the TMS320C6713 DSP processor and Code Composer Studio (CCS). Real-time audio input and output are handled using the AIC23 audio codec, interfaced through the aic23.h library. The codec captures live audio input through a microphone or line-in, converts it to digital form, and sends it to the DSP for processing. After applying interpolation or decimation along with FIR filtering, the processed audio is sent back to the codec and output through speakers or headphones with minimal delay.

This real-time implementation demonstrates the practical application of DSP concepts in embedded systems, highlighting how theoretical signal processing techniques can be used in live audio scenarios. The project is highly relevant to music playback devices, speech processing systems, audio effects units, and multimedia applications, and serves as an effective educational platform for understanding real-time multirate DSP and audio codec interfacing using CCS.

OBJECTIVE:

- To design and implement an audio speed control system using digital signal processing techniques.
- To increase audio playback speed by applying decimation while minimizing aliasing effects.
- To reduce audio playback speed using interpolation techniques with proper sample rate enhancement.
- To employ an FIR low-pass filter to maintain audio quality by suppressing unwanted frequency components.
- To demonstrate real-time DSP implementation of the audio speed control system on the TMS320C6713 DSP processor.

BLOCK DIAGRAM

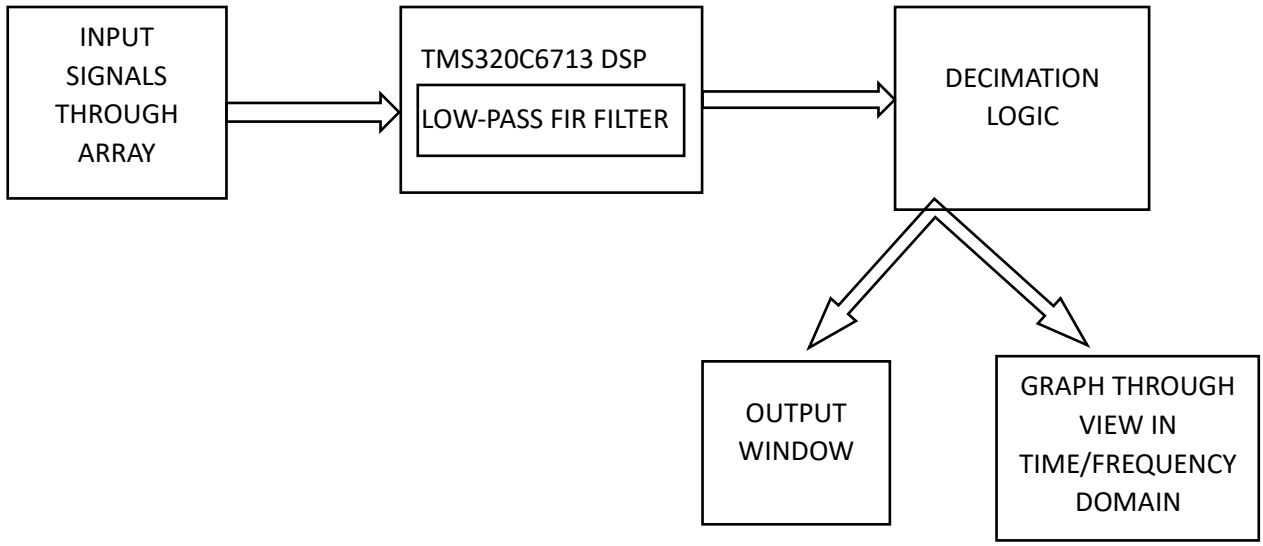


FIGURE (a): Decimation

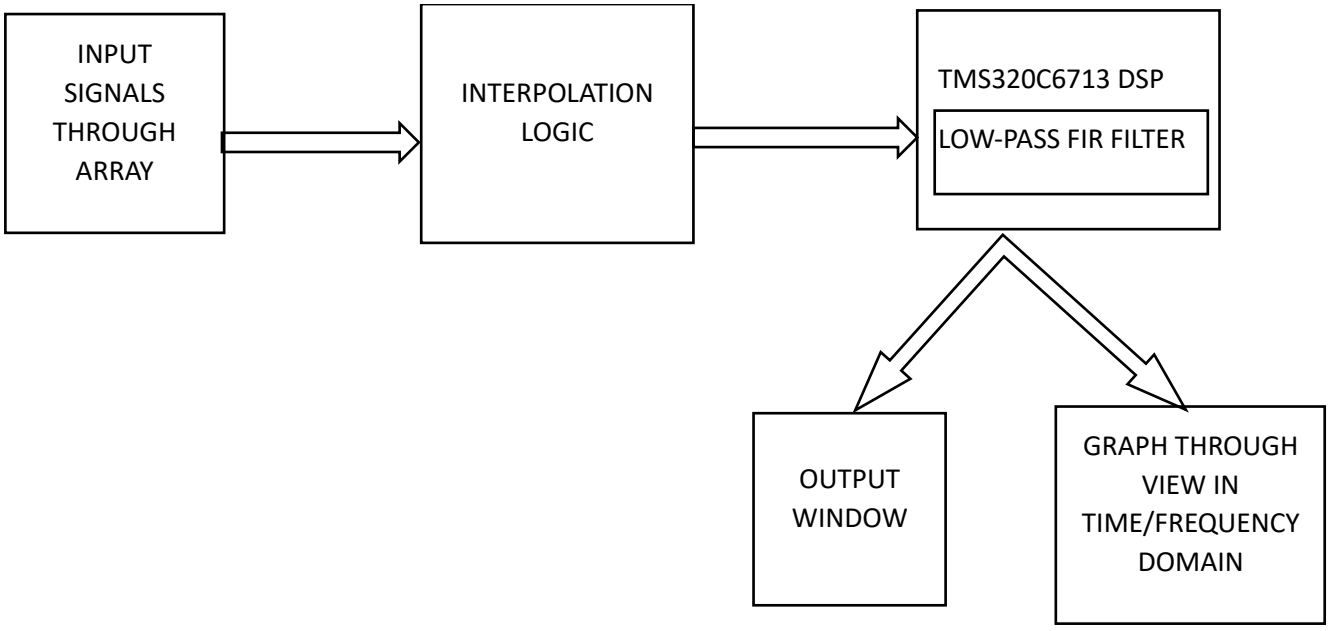


FIGURE (b): Interpolation

The diagram shows the audio speed control system implemented using decimation and interpolation on the TMS320C6713 DSP processor.
In decimation, the input signal is first passed through a low-pass FIR filter to avoid aliasing, and then the sampling rate is reduced, resulting in faster audio playback.
The decimated output is displayed in the output window and analyzed in both time and frequency domains.
In interpolation, the input signal is first up-sampled using interpolation logic by inserting zeros between samples.
The interpolated signal is then passed through a low-pass FIR filter to remove spectral images.
Finally, the processed output is observed through graphical analysis in the time and frequency domains.

EXPLANATION

This project demonstrates audio speed control using digital signal processing techniques, specifically decimation and interpolation, along with basic frequency-domain analysis. The given C program illustrates how changing the sampling rate of a discrete-time signal affects audio playback speed. The implementation is intended for real-time DSP environments such as the TMS320C6713 DSP processor, commonly programmed using Code Composer Studio.

The input signal is defined as an array of 8 samples, representing a discrete-time audio signal. The decimation factor M = 2 and interpolation factor L = 2 are selected to clearly demonstrate both speed-up and slow-down operations. Macros are used to define signal lengths and parameters, ensuring flexibility and clarity in the program.

Decimation is implemented by selecting every Mth sample from the input signal. This is achieved using a loop that increments the index by the decimation factor. By reducing the number of samples, the signal duration becomes shorter, which corresponds to faster audio playback. This method directly demonstrates how down-sampling increases playback speed in digital audio systems.

Interpolation is implemented by first initializing the output array to zero and then inserting the original samples at positions spaced by the interpolation factor L. This zero-insertion process increases the number of samples and expands the signal in time, resulting in slower audio playback. In practical DSP

systems, a low-pass FIR filter is applied after zero insertion to remove spectral images and smooth the signal, which is an essential concept behind interpolation-based audio speed control.

To study the spectral characteristics of the input signal, the program computes the Discrete Fourier Transform (DFT). Separate arrays are used to store the real and imaginary components of the frequency-domain representation. Trigonometric functions are applied to calculate these components, and the magnitude spectrum is obtained using the square root of the sum of squares of the real and imaginary parts. This frequency-domain analysis helps in understanding how sampling rate changes influence the signal spectrum.

Finally, the program prints the input signal, decimated signal, and interpolated signal for verification and observation. An infinite loop is included at the end of the program to allow continuous execution, enabling real-time visualization of signals using CCS graph tools. Overall, the program provides a clear and practical demonstration of audio speed control using decimation and interpolation, along with fundamental DSP concepts such as sampling rate conversion and frequency analysis.

CODE (C language)

```
#include <math.h>

#include <stdio.h>

#define N 8

#define M 2

#define L 2

#define NI ((N-1)*L + 1)

#define PI 3.141592653589793

/* Input signal */

float input_signal[N] = {0,1,2,3,4,5,6,7};

/* Decimation & Interpolation */

float decimated_signal[N/M];

float interpolated_signal[NI];

/* FFT buffers */

float real[N];

float imag[N];
```

```
float magnitude[N];
int i, j, k;
void main(void){
   /* -------- DECIMATION -------- */
   for(i = 0, j = 0; i < N; i += M, j++){
      decimated_signal[j] = input_signal[i];
   }
   /* -------- ZERO INSERTION -------- */
   for(i = 0; i < NI; i++){
      interpolated_signal[i] = 0.0;
   }
   /* -------- INTERPOLATION -------- */
   for(i = 0; i < N; i++){
      interpolated_signal[i * L] = input_signal[i];
   }
   /* -------- COPY INPUT TO FFT REAL PART -------- */
   for(i = 0; i < N; i++){
      real[i] = input_signal[i];
      imag[i] = 0.0;
   }
   /* -------- DFT (Frequency Domain) -------- */
   for(k = 0; k < N; k++){
      real[k] = 0.0;
      imag[k] = 0.0;
      for(i = 0; i < N; i++){
         real[k] += input_signal[i] * cos(2 * PI * k * i / N);
         imag[k] -= input_signal[i] * sin(2 * PI * k * i / N);
      }
      magnitude[k] = sqrt((real[k] * real[k]) + (imag[k] * imag[k]));
```

```
    }
    /* -------- PRINT RESULTS -------- */
    printf("Input Signal:\n");
    for(i = 0; i < N; i++){
        printf("%f ", input_signal[i]);
    }
    printf("\n\nDecimated Signal:\n");
    for(i = 0; i < N/M; i++){
        printf("%f ", decimated_signal[i]);
    }
    printf("\n\nInterpolated Signal:\n");
    for(i = 0; i < NI; i++){
        printf("%f ", interpolated_signal[i]);
    }
    printf("\n\nProgram Completed\n");
    /* Infinite loop for CCS graph */
    while(1);
}
```

STEPS OF EXECUTION

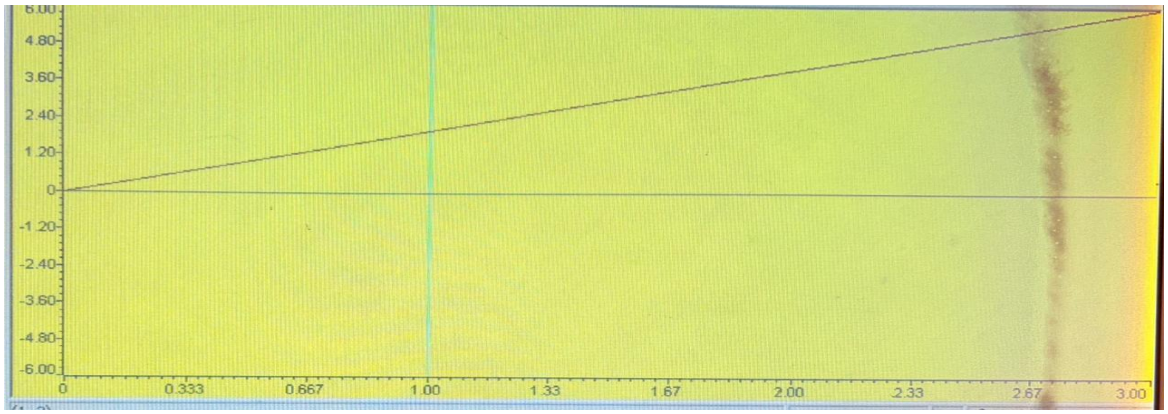
1. Create a new project in Code Composer Studio with target set to TMS320C67XX.
2. Configure the C6713 DSK and select the correct board support files.
3. Write the C program for audio sampling using line-in of the DSK.
4. Implement Butterworth low-pass, band-pass, and high-pass filters in C.
5. Read the audio samples from ADC and pass them through the filter bank.
6. Calculate interpolated and decimated output by theoretical calculations.
7. Compile the program.
8. Build and load the program onto the TMS320C6713 board.
9. Run the program
10. Compare that result with the practical result
11. Observe real-time output on graphs of CCS output window.

OUTPUT:

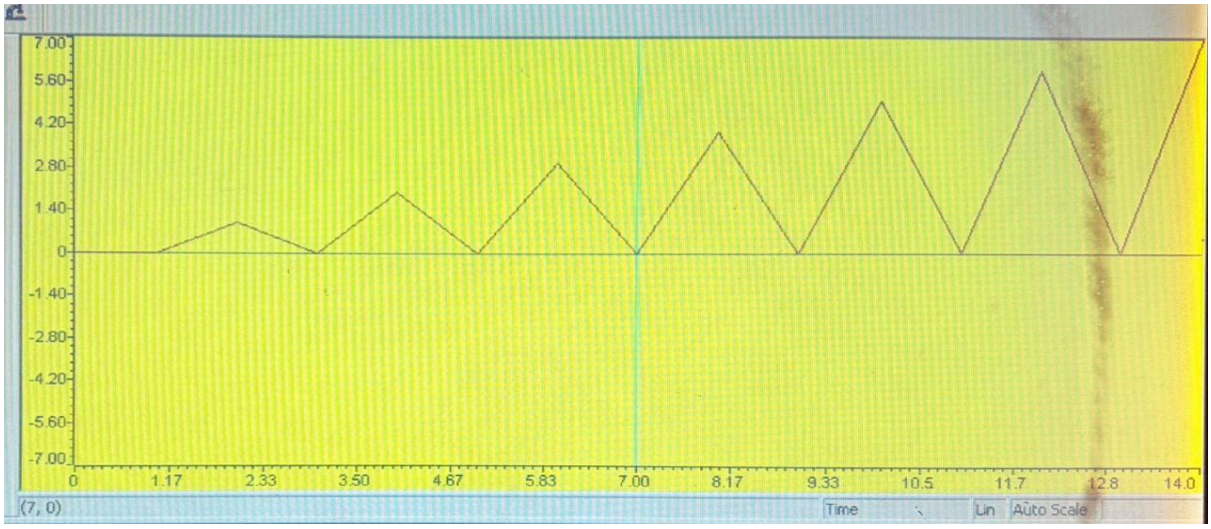
```
Program Completed.
Input Signal:
0.000000 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000

Decimated Signal:
0.000000 2.000000 4.000000 6.000000

Interpolated Signal:
0.000000 0.000000 1.000000 0.000000 2.000000 0.000000 3.000000 0.000000 4.000000

Program Completed
```

GRAPH:

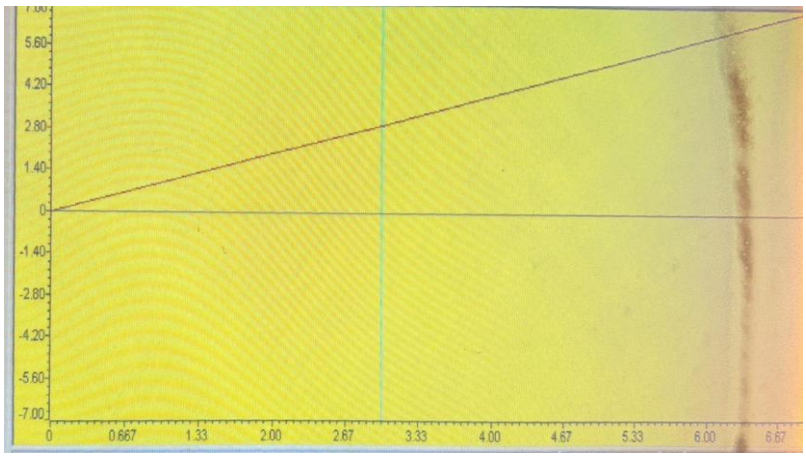
INPUT SIGNAL



INTERPOLATED SIGNAL



DECIMATED SIGNAL



RESULT:

Input Signal:
The input signal is a uniformly increasing discrete sequence. In the time-domain graph, it appears as a smooth, steadily rising waveform, representing the original audio signal sampled at a fixed rate.

Decimated Signal (Speed-Up):
The decimated signal is obtained by selecting every second sample of the input signal (M = 2). Compared to the input signal, the number of samples is reduced and the waveform becomes compressed along the time axis. This indicates that the signal completes the same amplitude variation in a shorter time, which corresponds to faster audio playback. Although the general trend of the input signal is preserved, fewer data points are present, clearly showing the effect of down-sampling**.**

Interpolated Signal (Slow-Down):
The interpolated signal is generated by inserting one zero between every two samples of the input signal (L = 2). When compared to the input signal, the interpolated waveform is stretched along the time axis and contains additional points, causing the signal to vary more slowly. This represents slower audio playback. The zero-inserted samples introduce visible gaps in the waveform, which in practical systems are smoothed using a low-pass FIR filter.

APPLICATIONS:

1. **Audio Playback Systems**
   Used to adjust playback speed in media players.
2. **Speech Processing**
   Helpful for slow-down or speed-up of speech for learning or analysis.
3. **Music Practice Tools**
   Enables musicians to slow down tracks without changing pitch.
4. **Hearing Aid Devices**
   Used for time-scale modification of audio signals.
5. **Multimedia Devices**
   Applied in video-audio synchronization systems.
6. **Digital Audio Effects**
   Used in sound effects and audio editing software.
7. **DSP Education and Labs**
   Widely used to teach interpolation, decimation, and filtering concepts.
8. **Communication Systems**
   Sampling rate conversion in digital communication receivers.
9. **Voice Recording and Editing**
   Useful for adjusting narration speed.
10. **Embedded Audio Systems**
    Implemented in DSP-based embedded products for audio control.

ADVANTAGES

1. **Flexible Audio Speed Control**
   Allows both speed-up and slow-down of audio using decimation and interpolation.
2. **Preserves Audio Quality**
   Use of FIR low-pass filtering minimizes aliasing and imaging effects.
3. **Linear Phase Response**
   FIR filters maintain waveform shape, which is important for audio signals.
4. **Real-Time Processing Capability**
   Can be implemented in real time on DSP processors like TMS320C6713.
5. **Simple Mathematical Implementation**
   Interpolation and decimation are easy to understand and implement.

6. **Low Computational Complexity**
   Efficient for small and medium sampling rates.
7. **Educational Value**
   Demonstrates key DSP concepts such as sampling rate conversion and DFT.
8. **Scalable Design**
   Speed change factor can be easily modified by changing M and L values.
9. **Software-Based Solution**
   No additional hardware is required for speed control.
10. **Wide DSP Compatibility**
    Can be implemented on various DSP platforms and processors.

DISADVANTAGES

1. **Aliasing in Decimation**
   Improper filtering may cause distortion due to aliasing.
2. **Imaging in Interpolation**
   Zero insertion introduces unwanted spectral images.
3. **Increased Processing Delay**
   FIR filtering introduces latency in real-time systems.
4. **Higher Memory Usage**
   Interpolation increases the number of samples to be processed.
5. **Limited Speed Control Accuracy**
   Speed change is restricted to integer factors (M and L).

CONCLUSION

The Audio Speed Control System demonstrates the effective use of interpolation and decimation to modify audio playback speed. Decimation speeds up the audio by reducing the sampling rate, while interpolation slows it down by increasing the sampling rate. The use of an FIR low-pass filter minimizes aliasing and imaging, ensuring good audio quality. Real-time implementation on the TMS320C6713 DSP processor confirms the practicality of multirate DSP techniques for real-time audio applications.